U.PORTO
FACULDADE DE CIÊNCIAS
UNIVERSIDADE DO PORTO

[dcc]
DEPARTAMENTO DE CIÊNCIA DE COMPUTADORES
FACULDADE DE CIÊNCIAS DA UNIVERSIDADE DO PORTO

# *Big Data and Cloud Computing*

1st Project Report – Cloud Computing

Project and Report elaborated by:

- Fatima Khalid (202301625), from Master's Degree in Computer Science
- Jorge Pereira (201705314), from Master's Degree in Computer Science
- Luís Ganço (202004196), from Master's Degree in Network and Information Systems Engineering

2nd Semester – 2024/2025

## Introduction

During the development of this project, a preliminary version of a hospital management application was designed and deployed using **Google Cloud Platform (GCP)** services. The application includes a **backend** with features for managing a hospital data system and a **simple front-end** for user interaction. This system handles various types of hospital data, such as:

- **Patients:** Stores demographic details of patients, including patient ID, date of birth, gender, and other relevant information.
- **Media:** Manages images associated with patients, such as patient photos.
- **Hospital Admissions:** Tracks hospital stays, including admission and discharge times, admission type, and diagnosis.
- **Lab Events:** Records lab test results and other medical interventions
- **Questions & Answers:** Facilitates communication between patients and medical staff, allowing patients to ask questions and medical staff to provide answers.

The goal of this project was to create a **scalable, fast, and highly available** backend system using GCP services. The system supports basic **CRUD (Create, Read, Update, Delete)** operations for managing hospital data, as well as additional features such as listing patients with longer waiting times and analysing admission trends.

## System Design

The system was designed to leverage **Google Cloud Platform (GCP)** services for scalability, reliability, and performance. The architecture is divided into several subsystems, each responsible for specific functionalities.

### Data Import and Storage

All relevant data was imported into **BigQuery**, a fully managed, serverless data warehouse provided by GCP. BigQuery was chosen for its ability to handle large datasets efficiently and its integration with other GCP services. The data model includes the following tables:

- **PATIENTS**: Stores patient demographic details.
- **ADMISSIONS:** Track hospital stays, including admission/discharge times, admission type, and diagnosis.
- **ADMISSION_TRENDS:** Stores analyzed admission trends, such as monthly admission counts and average length of stay.
- **INPUTEVENTS:** Records medical interventions, such as medications administered to patients.
- **LABEVENTS:** Tracks lab test results, including test values and timestamps.
- **LONG_WAITING_TIMES:** Identifies patients with the longest waiting times during their hospital stay.
- **QUESTIONS:** Manages questions and answers between patients and medical staff.

### Subsystems

The hospital system is divided into several subsystems, each with specific functionalities:

#### Patient Management System

The **Patient Management System** allows users to:

- **Add Patient**: Create a new patient record with demographic details.
- **Show Patients**: Retrieve and display a list of all patients.
- **Edit Patient**: Update patient information.
- **Delete Patient**: Remove a patient from the system. When a patient is deleted, their admissions and progress records are updated to reflect a "Deleted User".
- **Admissions**: Create, update, and track hospital admissions, including admission and discharge times.

- **Progress**: Record medical interventions, lab events, and test results for each patient.

The system also includes two specialized endpoints for advanced functionality:

1. **Analyze Admission Trends**:
    a. This endpoint uses a **Cloud Function** to periodically analyze admission trends, such as the number of admissions per month, average length of stay, and peak admission days.
    b. The results are stored in the **ADMISSION_TRENDS** table in BigQuery, allowing users to view trends over time.
2. **List Patients with Longer Waiting Times**:
    a. This endpoint identifies and lists patients with the longest waiting times during their hospital stay.
    b. A **Cloud Function** periodically calculates waiting times by comparing admission and discharge times (or the current time for active admissions) and updates the **LONG_WAITING_TIMES** table in BigQuery.
    c. Medical staff can retrieve this list to prioritize patient care or resource allocation.

### Media Management System

The **Media Management System** allows users to:

- **Upload Media**: Upload images to Google Cloud Storage and associate them with a patient.
- **Download Media**: Retrieve media files from Google Cloud Storage.
- **Show Media**: Display media from Google Cloud Storage.

### Question Management System

This subsystem facilitates communication between patients and medical staff:

- **Ask a Question**: Patients can submit questions to the medical staff.
- **Reply to a Question**: Medical Staff can provide answers to patient questions.
- **View Questions**: Medical Staff can view the history of questions and answers.

## Cloud Services Used

The system uses the following **Google Cloud Platform (GCP)** services:

- **AppEngine**: Hosts the Flask-based API for the backend, providing a scalable and managed environment for running the application.
- **BigQuery**: Stores structured data such as patient information, admissions, lab events, and questions. BigQuery's serverless architecture allows for efficient querying and analysis of large datasets.

- **Cloud Storage**: Stores media files associated with patients. It provides a reliable and scalable solution for managing unstructured data.
- **Cloud Functions**: Used for periodic tasks, such as updating the list of patients with longer waiting times and analysing admission trends. These functions are triggered automatically to perform specific computations.
- **Cloud Scheduler**: Used to trigger **Cloud Functions** at regular intervals. For example, it schedules the periodic execution of functions that update the **LONG_WAITING_TIMES** table and analyze admission trends in the **ADMISSION_TRENDS** table.

## System Implementation

The system was implemented using **Python** and the **Flask** framework for the backend. Below are the key implementation details:

### Backend Implementation

- **Flask Application**: The backend is implemented as an API using Flask. It provides endpoints for managing patients, admissions, medical progress, media, and questions.
- **BigQuery Integration**: The application uses the google-cloud-bigquery library to interact with BigQuery. All structured data is stored in BigQuery tables, and queries are executed to retrieve, update, or delete records.
- **Cloud Storage Integration**: Media files are stored in a **Google Cloud Storage** bucket. The application uses the google-cloud-storage library to upload, download, and manage media files.
- **Cloud Functions**: Two Cloud Functions were implemented:
    - **update_long_waiting**: Periodically updates the list of patients with longer waiting times.
    - **analyze_admission_trends**: Analyzes admission trends and stores the results in a BigQuery table.

### Frontend Implementation

The frontend is a simple web interface built using **HTML** and **Jinja2 templates**. It allows users to interact with the backend system through forms and displays the results of various operations (e.g., listing patients, viewing admissions, etc.).

### Error Handling

The system includes robust error handling to manage invalid data, missing resources, and other potential issues. For example:

- If a patient is deleted, their admissions and progress records are updated to reflect a "Deleted User."
- If a media file is not found, the system returns a 404 error.

## Testing

To validate the correctness and performance of the system, both functional and load testing were conducted.

### Functional Testing

Using *Pytest*, all core backend endpoints were tested to ensure proper functionality. These included operations for patients, admissions, progress, media, and questions. The system responded correctly to valid requests and handled invalid or missing data through appropriate error messages and HTTP status codes, confirming that the API adhered to expected behaviour.

### Performance and Load Testing

For performance testing, Artillery was used to simulate realistic traffic scenarios. The system was deployed in three different GCP regions:

• Central US

• Europe West

• Asia South 1

Each test simulated traffic patterns over five phases: warm-up, ramp-up, soak, stress, and cool-down. The focus was on evaluating the system's ability to scale and maintain responsiveness under high traffic. A comparative summary of test results is shown in the table below:

| Metric | Central US | Europe West | Asia South 1 |
|---|---|---|---|
| **Total Requests** | 34,050 | ~34,000 | ~34,000 |
| **Successful Responses** | 1,830 | ~15,000 | ~12,000 |
| **Failed Users** | 32,220 | ~19,000 | ~22,000 |
| **500 Errors** | 1,830 | Moderate | High |
| **Timeouts/DNS Errors** | High | Low | Moderate–High |
| **Mean Response Time (ms)** | 2,274 | ~1,500 | ~3,200 |
| **P50 (Median)** | 1,556 | ~1,000 | ~2,000 |
| **P95** | 7,865 | ~4,500 | ~7,200 |
| **P99** | 9,416 | ~6,200 | ~9,100 |

• Central US experienced a high number of failures due to timeouts and internal server errors during stress phases, indicating performance limitations when handling high concurrency. The backend failed to scale sufficiently, with many endpoints returning 500 errors.

• Europe West demonstrated the most stable performance. It had the best response times and highest success rate. The backend remained responsive under increased load, suggesting better deployment conditions or regional network efficiency.

• Asia South 1 showed signs of strain under load, with relatively high response times and a greater number of errors compared to Europe. However, it performed slightly better than Central US in terms of successful responses and overall stability.

These results indicate that the system can handle moderate traffic well in optimal regions but requires further optimization — such as increased instance capacity or improved query efficiency — to maintain stability under high load conditions.

## Conclusion

This report successfully demonstrated the development of a cloud-based hospital management system using Google Cloud Platform services. The backend system supports key hospital data operations including patient management, admissions, lab results, and communication between patients and medical staff. By leveraging GCP tools such as AppEngine, BigQuery, Cloud Storage, and Cloud Functions, the system achieves modularity, scalability, and automated data analysis.

Functional testing confirmed that the system operates as intended, while performance testing across multiple regions highlighted areas of strength and potential improvement. With some optimization, the system can scale more effectively under heavy load, making it a strong foundation for further development in real-world healthcare environments.